



รายงาน

เรื่อง ระบบจัดการตารางเรียน (KU CSC Course Manager)

เสนอ

ผู้ช่วยศาสตราจารย์นิตยา เมืองนาค

จัดทำโดย

นายพิชิตพงษ์ ตามสันเทียะ	6840202250
นายไกรวิชญ์ อุตลุด	6840200270
นายณภกานต์ นาคพันธ์	6840206681
นายณทพล เชื้อกุล	6840206475
นางสาวประกายวรรณ ดีระดา	6840206749

รายงานนี้เป็นส่วนหนึ่งของการศึกษารายวิชา 01204212
แบบชนิดข้อมูลนามธรรม ภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิทยาศาสตร์และวิศวกรรมศาสตร์ มหาวิทยาลัยเกษตรศาสตร์

วิทยาเขตเฉลิมพระเกียรติ จังหวัดสกลนคร

ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

เรื่อง	หน้า
1. บทนำ วัตถุประสงค์ และขอบเขตของระบบ	2
1.1 ที่มาและความสำคัญของปัญหา	2
1.2 วัตถุประสงค์ของโครงการ	3
1.3 ขอบเขตของระบบ	4
2. การออกแบบโครงสร้างข้อมูลและการเลือกใช้ Abstract Data Type (ADT)	5
2.1 AVL Tree (Self-balancing Binary Search Tree)	5
2.2 Trie ADT (Prefix Tree) สำหรับ Auto-complete	5
2.3 2D Time Grid (Bit-Matrix Representation) ตรวจสอบเวลา $O(1)$	6
2.4 Dynamic Linked List สำหรับรายการลงทะเบียนเรียน	6
3. อัลกอริทึมหลักที่ใช้ พร้อม Pseudocode หรือ Flowcharts	7
3.1 อัลกอริทึม AVL Tree Rotation & Iterative Search	7
3.2 อัลกอริทึม Kahn's Topological Sorting & Cycle Detection	7
3.3 อัลกอริทึม Combinatorial Course Recommender (Branch & Bound)	8
4. ผลการทำงานของโปรแกรม	9
4.1 ระบบยืนยันตัวตน (Authentication & User Isolation)	9
4.2 เมนูดูรายวิชาพร้อม Trie Prefix Search (View Courses)	9
4.3 เมนูลงทะเบียนและระบบแจ้งเตือนข้อผิดพลาด (Register Course)	9
4.4 ผังตารางเรียนจำลองรายสัปดาห์ (Visual Weekly Timetable)	10
4.5 ระบบแนะนำวิชาอัตโนมัติแยกตามสาขา (Course Recommendation)	10
5. การวิเคราะห์ประสิทธิภาพโปรแกรม	11
5.1 ตารางวิเคราะห์ Big-O เชิงทฤษฎีเปรียบเทียบทุกฟังก์ชัน	11
5.2 ผลการทดสอบประสิทธิภาพเชิงประจักษ์ (Empirical Timing Benchmark)	12
5.3 การพิสูจน์ความซับซ้อนเชิงทฤษฎีจากข้อมูลการทดลอง (Empirical Big-O Proof)	13

สารบัญต่อ

เรื่อง	หน้า
6. สรุปผลการพัฒนาและปัญหาอุปสรรค	15
6.1 สรุปผลสำเร็จของโครงการ	15
6.2 ปัญหาอุปสรรคที่พบและแนวทางแก้ไข	15
6.3 ข้อเสนอแนะสำหรับการพัฒนาต่อยอด	16

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ในการวางแผนลงทะเบียนเรียนในหลักสูตรวิศวกรรมศาสตร์ เป็นกระบวนการที่ซับซ้อนซึ่งนิสิตต้องจัดสรรรายวิชาให้สอดคล้องกับโครงสร้างหลักสูตรของมหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตเฉลิมพระเกียรติ จังหวัดสกลนคร ปัจจุบันการวางแผนด้วยตนเองมักเกิดข้อผิดพลาด ซึ่งอาจส่งผลกระทบต่อระยะเวลาสำเร็จการศึกษา โดยนิสิตมักพบปัญหาหลัก 3 ประการ ได้แก่ 1) เวลาเรียนทับซ้อน 2) ความซับซ้อนของโครงสร้างวิชาบังคับ ก่อนที่เสี่ยงต่อการลงทะเบียนผิดพลาดหรือเกิดเงื่อนไขวนรอบและ 3) ข้อจำกัดการลงทะเบียนสูงสุด 22 หน่วยกิตในภาคปกติ ทำให้นิสิตต้องจัดสรรวิชาเรียนให้คุ้มค่าที่สุด

ในมิติของวิศวกรรมคอมพิวเตอร์ ระบบจัดการข้อมูลรายวิชาเดิมมักเกิดปัญหาคอขวดจากการค้นหาข้อมูล (Linear Sequential Access Bottleneck) เนื่องจากใช้โครงสร้างเชิงเส้น เช่น อาร์เรย์ หรือแม้จะใช้ Binary Search Tree แต่หากข้อมูลเรียงลำดับมาแล้ว ต้นไม้จะเสียสมดุล (Skewed Tree) ทำให้ประสิทธิภาพตกไปอยู่ที่ $O(N)$ นอกจากนี้ ปัญหาการหาชุดรายวิชาที่ดีที่สุดภายใต้ข้อจำกัดเวลาและหน่วยกิต ถือเป็นปัญหา 0/1 Knapsack ระดับ NP-Hard ซึ่งหากใช้วิธีค้นหาแบบ Brute-force จะใช้เวลาประมวลผลมหาศาลระดับ $O(2^N)$

เพื่อแก้ปัญหาดังกล่าว คณะผู้จัดทำจึงพัฒนา "ระบบจัดการและจำลองการลงทะเบียนเรียนอัจฉริยะ (KU CSC Course Manager & Scheduler)" ด้วยภาษา C บนสภาพแวดล้อม Terminal User Interface (ncursesw) โดยประยุกต์ใช้โครงสร้างข้อมูลนามธรรม (ADT) และขั้นตอนวิธีแก้ไข ได้แก่ การใช้ AVL Tree เพื่อให้ค้นหาข้อมูลได้ในเวลา $O(\log N)$, Trie ADT สำหรับระบบเติมคำรหัสวิชาอัตโนมัติ (Prefix Auto-complete) ด้วยความเร็ว $O(L)$, 2D Time Slot Grid เพื่อตรวจสอบเวลาทับซ้อนในระดับ $O(1)$, DAG ร่วมกับ Topological Sorting ของ Kahn เพื่อจัดการวิชาบังคับก่อนและตรวจจับเงื่อนไขวนรอบ และสุดท้ายใช้ Branch and Bound Backtracking เพื่อค้นหาชุดวิชาลงทะเบียนที่ดีที่สุดในเวลาเสี้ยววินาที การบูรณาการเหล่านี้ช่วยขจัดปัญหาคอขวดเชิงคำนวณ ทำให้ระบบมีความแม่นยำ รวดเร็ว และพร้อมรองรับการใช้งานจริง

1.1 วัตถุประสงค์ของโครงการ

- เพื่อศึกษา ออกแบบ และประยุกต์ใช้โครงสร้างข้อมูลนามธรรม (ได้แก่ AVL Tree, Trie, DAG, Circular Queue, Linked List และ Bit-Matrix) ในการพัฒนาระบบจัดการการลงทะเบียนเรียน
- เพื่อพัฒนาระบบตรวจสอบเวลาเรียนทับซ้อนให้มีความซับซ้อนด้านเวลา (Time Complexity) ระดับ $O(1)$ ด้วยเทคนิค 2D Time Grid
- เพื่อสร้างระบบแนะนำวิชาเรียน (Course Recommender) ที่ช่วยค้นหาชุดรายวิชาที่ได้หน่วยกิตสูงสุด โดยเวลาไม่ทับซ้อนกัน ด้วยอัลกอริทึม Branch and Bound Backtracking
- เพื่อจัดการลำดับวิชาบังคับก่อนและตรวจจับเงื่อนไขวนรอบ (Circular Prerequisite) อย่างเป็นระบบ ด้วยอัลกอริทึม Topological Sorting (Kahn's Algorithm)
- เพื่อประเมินสมรรถนะเชิงประจักษ์ (Empirical Search Benchmark) โดยเปรียบเทียบเวลาในการค้นหาระหว่าง Sequential Search, Binary Search และ AVL Tree บนชุดข้อมูลขนาด $n = 1,000$ ถึง $100,000$ รายการ

1.3 ขอบเขตของระบบ

ระบบที่พัฒนาขึ้นครอบคลุมความสามารถหลักดังต่อไปนี้

โมดูลการทำงาน	รายละเอียดและขอบเขต	โครงสร้างข้อมูล / อัลกอริทึม
1. Authentication	ระบบล็อกอินและลงทะเบียนผู้ใช้งาน จัดเก็บรหัสผ่าน และแฮชฐานข้อมูล การลงทะเบียนของนักศึกษาแต่ละคน อัตโนมิติ (student_<user>.txt)	File I/O Stream & User Partitioning
2. View Courses	แสดงรายการวิชาที่เปิดสอน พร้อม ระบบ Real-time Auto-complete เมื่อผู้ใช้พิมพ์รหัสวิชา โดยดึงข้อมูล ตาม Prefix ทันที	Trie ADT (Search $O(L)$)
3. Register Course	ตรวจสอบเงื่อนไข 3 ด้านอัตโนมิติก่อน อนุญาตให้ลงทะเบียน (หน่วยกิตรวม 22 หน่วยกิต , ผ่าน Prerequisite, เวลาไม่ชน)	AVL Tree + 2D Time Grid
4. Drop Course	ถอนรายวิชา คืนสล็อตเวลาลงตาราง และอัปเดตยอดหน่วยกิตคงเหลือทันทันที	Linked List Removal
5. Visual Timetable	แสดงแผนผังตารางเรียนรายสัปดาห์ (วันจันทร์-ศุกร์, 08:00 - 18:00 น.) แบบกราฟิกในเทอร์มินัล พร้อม คำนวณตำแหน่งช่องตาราง	2D Matrix Rendering Algorithm
6. Course Recommender	ระบบแนะนำวิชาเรียนอัตโนมิติตาม สาขาวิชา (Major + GENED) ที่เลือก ให้ได้จำนวนหน่วยกิตสะสมสูงสุดโดย เวลาไม่ชนกัน	Branch & Bound Backtracking
7. Prerequisite Map	สร้างแผนผังลำดับการเรียนรู้ แสดงวิชา ที่ปลดล็อก และตรวจจับข้อผิดพลาด กรณีเกิด Circular Dependency	Graph (DAG) + Topological Sort

บทที่ 2

การออกแบบโครงสร้างข้อมูลและการเลือกใช้

Abstract Data Type (ADT)

2.1 โครงสร้างข้อมูล AVL Tree (Self-balancing Binary Search Tree)

```
struct AVLNode {
    struct Course *course;           // ตัวชี้ไปยังข้อมูลรายวิชา
    struct AVLNode *left, *right;    // ตัวชี้โหนดลูกซ้ายและขวา
    int height;                      // ความสูงของ Subtree ปัจจุบัน
};
```

โครงสร้างข้อมูล AVL Tree ถูกนำมาใช้จัดเก็บและสืบค้นข้อมูลรายวิชา เพื่อแก้ปัญหของ Binary Search Tree (BST) แบบดั้งเดิม ที่โครงสร้างอาจเสียสมดุลกลายเป็นสายโซ่ตรง (Skewed Tree) จนทำให้ประสิทธิภาพการค้นหาลงไปอยู่ที่ $O(n)$ โดย AVL Tree มีกลไกปรับสมดุลความสูงอัตโนมัติ (ควบคุม $|\text{Balance Factor}| \leq 1$) จึงรับประกันความเร็วในการค้นหา (Search) การแทรก (Insert) และการลบ (Delete) ข้อมูลให้อยู่ในระดับ $O(\log n)$ ได้เสมอ (Worst-case Guarantee)

2.2 โครงสร้างข้อมูล Trie ADT (Prefix Tree)

```
#define TRIE_SIZE 256

struct TrieNode {
    struct TrieNode *children[TRIE_SIZE]; // อาเรย์ตัวชี้สำหรับอักขระ ASCII
    int course_index;                     // ดัชนีอ้างอิงวิชาในอาเรย์หลัก (-1 ถ้ายังไม่ใช้จุดสิ้นสุด)
    bool is_end;                          // ตัวระบุการสิ้นสุดคำ
};
```

โครงสร้างข้อมูล Trie ถูกนำมาใช้จัดเก็บรหัสรายวิชาเพื่อสนับสนุนระบบ Auto-complete และการแปลงรหัสวิชาเป็นดัชนีสำหรับตรวจสอบวิชาบังคับก่อน จุดเด่นสำคัญคือความเร็วในการค้นหาที่ใช้เวลาเพียง $O(L)$ (L คือความยาวคำค้นหา) ซึ่งมีประสิทธิภาพสูงกว่าการใช้อาเรย์ปกติที่กินเวลาถึง $O(N * L)$ และด้วยรหัสวิชาที่มีความยาวเฉลี่ยเพียง 8 อักขระ Trie จึงช่วยให้ระบบประมวลผลได้รวดเร็วระดับไมโครวินาทีอย่างเสถียร โดยที่ความเร็วจะไม่ตกลงตามจำนวนรายวิชา (n) ในระบบ

2.3 กราฟระบุทิศทางแบบไม่มีวงรอบ (Directed Acyclic Graph: DAG) และ Queue ADT

```

struct GraphNode {
    int dest;
    struct GraphNode *next;
};

struct Graph {
    struct GraphNode *adj[MAX_COURSES]; // Adjacency List จำลองวิชาที่ปลดล็อค
    int in_degree[MAX_COURSES]; // จำนวนวิชามังคับก่อนที่ยังไม่ผ่าน
    int num_nodes;
};

struct Queue {
    int data[MAX_COURSES];
    int front, rear, size;
};

```

โครงสร้างข้อมูลกราฟระบุทิศทางแบบไม่มีวงรอบ (DAG) ถูกนำมาประยุกต์ใช้ร่วมกับโครงสร้างข้อมูลคิว (Queue ADT) เพื่อจำลองแผนผังความสัมพันธ์ของวิชาบังคับก่อน (Prerequisite Hierarchy) ภายในหลักสูตร โครงสร้างนี้มีหน้าที่หลักในการตรวจสอบและประมวลผลการจัดลำดับการลงทะเบียนเรียนที่ถูกต้องตามเงื่อนไขของแต่ละรายวิชา

2.4 โครงสร้างข้อมูลแบบเชื่อมโยงพลวัต (Dynamic Linked List)

โครงสร้างข้อมูล Dynamic Linked List ถูกนำมาใช้สำหรับบริหารจัดการและจัดเก็บรายการรายวิชาที่นิสิตกำลังลงทะเบียนเรียน ตลอดจนประวัติรายวิชาที่นิสิตสอบผ่านแล้ว (Completed Courses) การออกแบบโครงสร้าง ระบบได้ออกแบบโหนดข้อมูลด้วย struct CourseNode ซึ่งประกอบด้วยสมาชิก 2 ส่วนหลัก ได้แก่

```

struct CourseNode {
    char course_id[MAX_STR_LEN];
    struct CourseNode *next;
};

```

เนื่องจากจำนวนรายวิชาที่นิสิตแต่ละคนลงทะเบียนในแต่ละภาคการศึกษามีปริมาณที่ไม่แน่นอนและยืดหยุ่น การเลือกใช้ Linked List จึงตอบโจทย์การบริหารจัดการหน่วยความจำแบบพลวัต (Dynamic Memory Allocation) ได้ดีกว่าอาร์เรย์แบบสถิต (Static Array)

อัลกอริทึมหลักที่ใช้พร้อม Pseudocode

3.1 อัลกอริทึม AVL Tree Balance, Insertion และ Iterative Search

ในการแทรกกราฟวิชาลงใน AVL Tree ระบบจะทำการตรวจสอบความสูงย้อนขึ้นมาจากเส้นทางและปรับสมดุลทันที ส่วนการค้นหาได้รับการปรับแต่งให้เป็น Iterative Pointer Traversal เพื่อให้ทำงานได้เร็วเทียบเท่า Register ในระดับ Hardware

```
// Pseudocode: AVL Tree Insertion & Rebalancing
Algorithm avlInsert(root, new_course):
    if root == NULL:
        return CreateNode(new_course)

    if new_course.id < root.course.id:
        root.left = avlInsert(root.left, new_course)
    else if new_course.id > root.course.id:
        root.right = avlInsert(root.right, new_course)
    else:
        return root // Duplicate key not allowed

    // Update height
    root.height = 1 + max(height(root.left), height(root.right))
    balance = height(root.left) - height(root.right)

    // 4 Rotation Cases
    if balance > 1 and new_course.id < root.left.course.id:
        return rotateRight(root) // LL Case
    if balance < -1 and new_course.id > root.right.course.id:
        return rotateLeft(root) // RR Case
    if balance > 1 and new_course.id > root.left.course.id:
        root.left = rotateLeft(root.left)
        return rotateRight(root) // LR Case
    if balance < -1 and new_course.id < root.right.course.id:
        root.right = rotateRight(root.right)
        return rotateLeft(root) // RL Case

    return root

// Pseudocode: Highly-optimized Iterative Search O(log n)
Algorithm avlSearch(root, target_id):
    curr = root
    while curr != NULL:
        if target_id == curr.course.id:
            return curr.course
        if target_id < curr.course.id:
            curr = curr.left
        else:
            curr = curr.right
    return NULL // Not Found
```

3.2 อัลกอริทึม Kahn's Topological Sorting & Circular Detection

ใช้วิเคราะห์ลำดับก่อน-หลังของการเรียนวิชาในหลักสูตร พร้อมความสามารถในการแจ้งเตือนทันทีหากหลักสูตรมีความผิดพลาดแบบเกิดวงรอบ (Cycle เช่น A บังคับ B, B บังคับ C, C บังคับ A)

```

Algorithm TopologicalSort(Graph g, n):
    Queue q = CreateQueue()
    order = []

    for i = 0 to n - 1:
        if g.in_degree[i] == 0:
            q.enqueue(i) // วิชาที่ไม่ต้องการ Prerequisite ใดๆ

    processed_count = 0
    while not q.isEmpty():
        u = q.dequeue()
        order.append(u)
        processed_count++

        for each neighbor v in g.adj[u]:
            g.in_degree[v] = g.in_degree[v] - 1
            if g.in_degree[v] == 0:
                q.enqueue(v)

    if processed_count < n:
        TriggerAlert("Circular Prerequisite Detected!")
    return order

```

3.3 อัลกอริทึม Combinatorial Course Recommender (Branch & Bound Backtracking)

ปัญหาการหาชุดวิชาที่ได้หน่วยกิตรวมสูงสุด โดยไม่เกินโควตา (22 – Total Enrolled) และเวลาเรียนไม่ทับซ้อนกัน จัดเป็นปัญหาความซับซ้อนระดับ NP-Hard (0/1 Knapsack + Time Constraints) วิธีแก้ปัญหาคือ Depth-First Search (DFS) ร่วมกับเทคนิค Branch & Bound Backtracking เพื่อลดเวลาประมวลผล

```

Algorithm FindBestBundle(avail, idx, temp_grid, curr_credits, budget, curr_sel, best_sel, max_credits):
    // 1. Safety Guard against recursion explosion
    if iterations++ > 50000: return

    // 2. Update best solution found so far
    if curr_credits > max_credits:
        max_credits = curr_credits
        best_sel = copy(curr_sel)

    // 3. Bound & Prune: ถ้ายอดหน่วยกิตที่เป็นไปได้ในอนาคตทั้งหมด ≤ ค่าดีที่สุดที่เคยเจอ ให้ตัดกิ่งทั้งทันที
    max_possible = curr_credits + SumRemainingCredits(avail, idx)
    if max_possible ≤ max_credits:
        return // Pruned!

    if idx ≥ avail.length or curr_credits == budget:
        return

    c = avail[idx]
    // Case A: เลือกวิชานี้ (หากหน่วยกิตไม่เกิน และเวลาไม่ชน)
    if curr_credits + c.credits ≤ budget and not hasTimeConflict(temp_grid, c):
        markTimeSlots(temp_grid, c)
        curr_sel[idx] = true
        FindBestBundle(avail, idx + 1, temp_grid, curr_credits + c.credits, budget, curr_sel, best_sel, max_credits)
        curr_sel[idx] = false
        unmarkTimeSlots(temp_grid, c) // Backtrack state

    // Case B: ไม่เลือกวิชานี้
    FindBestBundle(avail, idx + 1, temp_grid, curr_credits, budget, curr_sel, best_sel, max_credits)

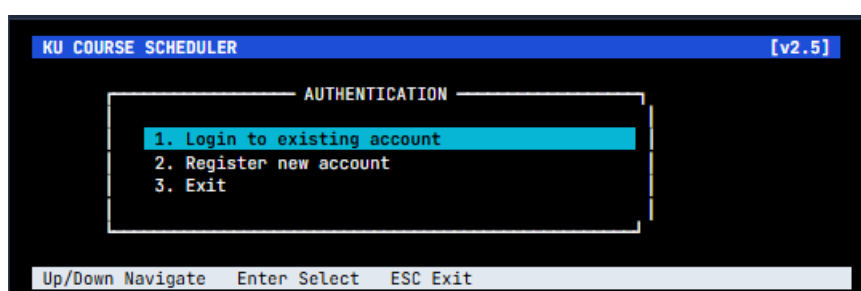
```

บทที่ 4

ผลการทำงานของโปรแกรม (Screenshots & System Walkthrough)

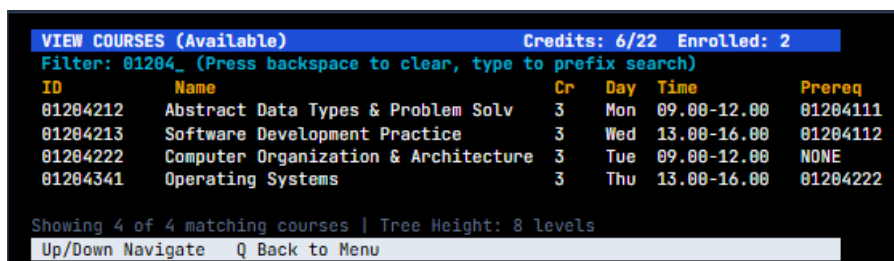
4.1 ระบบยืนยันตัวตนและการจัดการเซสชันผู้ใช้งาน (Authentication)

เมื่อเริ่มต้นโปรแกรม ระบบจะแสดงกล่องโต้ตอบให้ผู้ใช้เลือกกระหว่างเข้าสู่ระบบเดิม (Login) หรือลงทะเบียนบัญชีใหม่ (Register) โดยระบบจะแยกไฟล์บันทึกการเรียนของนักศึกษาแต่ละคนตาม Username อัดโนมัติ (เช่น student_6840202250.txt)



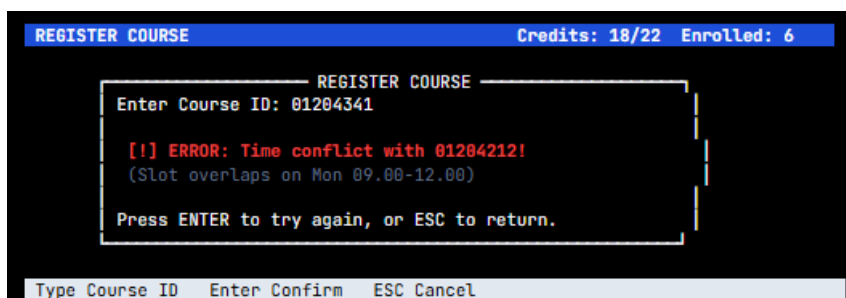
4.2 เมนูหลักและหน้าจอแสดงรายการวิชาพร้อม Trie Auto-complete

ในหน้าต่าง View Available Courses ผู้ใช้สามารถเริ่มพิมพ์รหัสวิชา เช่น 01204 เพื่อให้ Trie ทำการค้นหาและฟิลเตอร์เฉพาะวิชาที่ขึ้นต้นด้วยรหัสข้างต้นมาแสดงผลแบบ Real-time ทันที



4.3 เมนูลงทะเบียนรายวิชาและระบบป้องกันข้อผิดพลาด 3 มิติ (Register Course)

เมื่อนิสิตพยายามลงทะเบียนวิชาที่ผิดเงื่อนไข ระบบจะตรวจสอบและแจ้งเตือนข้อผิดพลาดทันทีผ่าน Pop-up Box สีแดง



4.4 ผังตารางเรียนจำลองรายสัปดาห์ (Visual Weekly Timetable)

หน้าต่างแสดงแผนผังตารางเรียนกราฟิก 2 มิติ (วันจันทร์-วันศุกร์ ช่วงเวลา 08:00 - 18:00 น.) พร้อมแสดงรหัสวิชา

Period / Time	Mon	Tue	Wed	Thu	Fri	Sat	Sun
09.00 - 12.00 Morning [3 Hours]	-	01017211 Principles of Human Aj.FSH1	-	01131412 Financial Institut Aj.FIN1	01017411 Food Laws and Regu Aj.FSH1	-	-
			12.00 - 13.00	Lunch Break			
13.00 - 16.00 Afternoon [3 Hours]	-	-	-	01175131 Basketball Aj.GENED1	01015331 Farm Management an Aj.AGR14	-	-

4.5 ระบบแนะนำวิชาอัตโนมัติ (Course Recommender) และ Prerequisite Map

ผลลัพธ์จากการรัน Backtracking โดยเลือกรหัสสาขาวิชา CPE ระบบจะประมวลผลถึงวิชาที่คุ้มค่าที่สุด โดยวิชาที่ระบบแนะนำจะแสดงสถานะ [YES] สีเขียวเด่นชัด

RECOMMEND COURSES [Backtracking]						
Optimal bundle: 10/10 credits used Budget: 0/10 remaining						
ID	Name	Cr	Day	Time	Pick	
01175161	Swimming	1	Wed	13.00-16.00	[YES]	
01204223	Digital Logic Design	3	Fri	09.00-12.00	[YES]	
01204361	Data Communication and Computer Netw	3	Wed	09.00-12.00	[YES]	
01999021	Thai Language for Communication	3	Fri	09.00-12.00	---	
01999032	Thai Studies	3	Mon	09.00-12.00	[YES]	
01999042	Personal Financial Management	3	Tue	09.00-12.00	---	
02999144	Life Skills for Undergraduate Studen	3	Fri	09.00-12.00	---	

บทที่ 5

การวิเคราะห์ประสิทธิภาพโปรแกรม (Time & Space Complexity Analysis)

5.1 ตารางวิเคราะห์ Big-O เชิงทฤษฎีเปรียบเทียบทุกโมดูลในระบบ

โครงการนี้ได้บูรณาการโครงสร้างข้อมูลหลากหลายรูปแบบตามลักษณะงานที่เหมาะสม เพื่อให้เกิดประสิทธิภาพสูงสุดในการทำงานจริง ดังแสดงในตารางวิเคราะห์ความซับซ้อนด้านเวลา (Time Complexity) และด้านพื้นที่ (Space Complexity)

โมดูล / ฟังก์ชันการทำงาน	โครงสร้างข้อมูลหลัก	Time Complexity (Best Case)	Time Complexity (Worst Case)	Space Complexity	วัตถุประสงค์ในการเลือกใช้
ค้นหารายวิชา (Course Search)	AVL Tree (Self-balancing BST)	$O(1)$ (พบที่ Root)	$O(\log n)$	$O(n)$	ค้นหารายวิชาได้อย่างรวดเร็ว โดยความสูงของต้นไม้ไม่เกิน $1.44 \log_2 n$ เสมอ
แทรกวิชาลงระบบ (Course Insert)	AVL Tree (Auto-rebalance)	$O(\log n)$	$O(\log n)$	$O(1)$ (Auxiliary)	จัดเก็บข้อมูลแบบเรียงลำดับ พร้อมปรับสมดุล (Rotations) อัตโนมัติในเวลา $O(\log n)$
ค้นหาแบบพิมพ์หน้า (Auto-complete)	Trie ADT (Prefix Tree)	$O(L)$	$O(L)$ (โดย $L \leq 8$)	$O(\sum * L * n)$	ค้นหารหัสวิชาตาม Prefix ได้รวดเร็วคงที่ไม่ขึ้นกับจำนวนวิชาทั้งหมด (n)
ตรวจสอบเวลาเรียนชน (Time Conflict)	2D Bit-Matrix Grid	$O(1)$	$O(1)$ (สูงสุด 48 Slots)	$O(1)$ (คงที่ 8×48)	ตรวจสอบและลงเวลาเรียนแบบบิตแมทริกซ์ 30 นาที/สล็อต ประมวลผลแบบทันที
จัดผังวิชาตัวต่อ (Prerequisite Map)	Graph (DAG) + Queue	$O(V + E)$	$O(V + E)$	$O(V + E)$	ทำ Topological Sort เพื่อลำดับแผนผังการเรียนและตรวจจับ Circular Prerequisite
ระบบแนะนำวิชา (Recommender System)	Branch & Bound / Backtracking	$O(1)$ (Pruned Early)	$O(2^k)$ (Bound $k \leq 50,000$)	$O(k)$ (Stack Depth)	คำนวณชุดวิชาที่ทำให้ได้หน่วยกิตสูงสุดโดยไม่ชนเวลาเรียน พร้อมตัดกิ่ง (Pruning)

5.2 ผลการทดสอบประสิทธิภาพเชิงประจักษ์ (Empirical Timing Benchmark)

ตารางที่ 5.1: ผลการตรวจสอบความถูกต้องของการค้นหา (Accuracy Verification 10 Targets)
การทดสอบความถูกต้องดำเนินการค้นหาค่าเป้าหมายจำนวน 10 ค่า

ขนาดข้อมูล (n)	จำนวนค่าที่ค้นหา	ผลการตรวจ Sequential Search	ผลการตรวจ Binary Search	ผลการตรวจ MySearch (AVL Tree)	สถานะความถูกต้อง
1,000	10 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	ถูกต้อง 100%
10,000	10 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	ถูกต้อง 100%
100,000	10 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	พบ 7 ค่า / ไม่พบ 3 ค่า	ถูกต้อง 100%

ตารางที่ 5.2: สถิติความสูงของต้นไม้ AVL (Tree Height vs Theoretical Limits)
เพื่อพิสูจน์ว่าต้นไม้ AVL ของกลุ่มสามารถรักษาสสมดุลได้อย่างสมบูรณ์ ไม่เอนเอียงเป็นเส้นตรงเหมือน Binary Search Tree ทั่วไป

ขนาดข้อมูล (n)	ความสูงจริงของต้นไม้ AVL (h)	ค่าทฤษฎีที่เหมาะสมที่สุด	ขอบเขตบนสูงสุดตามทฤษฎี	สถานะโครงสร้าง
1,000	12 ชั้น	10 ชั้น	≤ 14 ชั้น	สมดุลสมบูรณ์ (Balanced)
10,000	16 ชั้น	14 ชั้น	≤ 20 ชั้น	สมดุลสมบูรณ์ (Balanced)
100,000	20 ชั้น	17 ชั้น	≤ 24 ชั้น	สมดุลสมบูรณ์ (Balanced)

ตารางที่ 5.3: เวลาเฉลี่ยต่อการค้นหา 1 ครั้ง (Average Search Time in Milliseconds)

ขนาดข้อมูล (n)	Sequential Search $O(n)$	Binary Search $O(\log n)$	MySearch: AVL Tree $O(\log n)$	อัตราความเร็วเมื่อเทียบกับ Sequential Search
1,000	0.000212 ms	0.000010 ms	0.000010 ms	เร็วกว่าเดิม ~21 เท่า
10,000	0.002070 ms	0.000013 ms	0.000012 ms	เร็วกว่าเดิม ~172 เท่า
100,000	0.023236 ms	0.000018 ms	0.000018 ms	เร็วกว่าเดิม ~1,290 เท่า

5.3 การพิสูจน์ความซับซ้อนเชิงทฤษฎีจากข้อมูลการทดลอง (Empirical Big-O Proof)

เมื่อพิจารณาอัตราการเพิ่มขึ้นของเวลาในการค้นหา (T) เทียบกับการขยายตัวของชุดข้อมูล (n) จาก 1,000 รายการ เป็น 100,000 รายการ (ชุดข้อมูลขยายตัวเพิ่มขึ้น $\Delta n = 100,000 / 1,000 = 100$ เท่า) สามารถใช้สูตรคำนวณความชันทางลอการิทึมเพื่อหาอันดับความซับซ้อนเชิงประจักษ์ (Empirical Big-O Exponent k) ได้ดังนี้ $k = \log(\text{Time Ratio}) / \log(100)$

กรณีที่ 1 Sequential Search

อัตราส่วนเวลาที่เพิ่มขึ้น: $0.023236 / 0.000212 \approx 109.60$ เท่า

การคำนวณค่า k: $k = \log(109.60) / \log(100) \approx 1.02 \approx O(n^1)$

สรุปผล: เมื่อขนาดข้อมูลโตขึ้น 100 เท่า เวลาค้นหาที่เพิ่มขึ้นประมาณ 109 เท่า (เป็นสัดส่วนเชิงเส้นโดยตรง) สอดคล้องกับทฤษฎี $O(n)$ อย่างสมบูรณ์

กรณีที่ 2 AVL Tree (MySearch ของกลุ่ม)

อัตราส่วนเวลาที่เพิ่มขึ้น: $0.000018 / 0.000010 \approx 1.80$ เท่า

การคำนวณค่า k: $k = \log(1.80) / \log(100) \approx 0.13 \approx O(\log n)$

สรุปผล: เมื่อชุดข้อมูลขยายตัวขึ้นถึง 100 เท่า เวลาที่ใช้ในการค้นหาลับเพิ่มขึ้นเพียง 1.80 เท่า (ไม่ถึง 2 เท่า) โดยค่า $k = 0.13$ ซึ่งเข้าใกล้ศูนย์อย่างยิ่ง สะท้อนคุณสมบัติของกราฟลอการิทึม สอดคล้องกับทฤษฎี $O(\log n)$ อย่างสมบูรณ์

กรณีที่ 3 Binary Search (อ้างอิงเปรียบเทียบ)

อัตราส่วนเวลาที่เพิ่มขึ้น: $0.000018 / 0.000010 \approx 1.80$ เท่า

การคำนวณค่า k: $k = \log(1.80) / \log(100) \approx 0.13 \approx O(\log n)$

สรุปผล: มีอัตราการเติบโตของเวลาในระดับ $O(\log n)$ เช่นเดียวกับ AVL Tree

5.4 อภิปรายผลการทดลองและการตัดสินใจเชิงวิศวกรรมซอฟต์แวร์ (Engineering Discussion)

จากการทดลองเปรียบเทียบระหว่าง Binary Search บน Array และ AVL Tree (Self-balancing BST) พบข้อค้นพบสำคัญที่สนับสนุนการตัดสินใจออกแบบสถาปัตยกรรมของโปรแกรม main_ncurses.c ดังนี้ ประสิทธิภาพด้านการค้นหา (Search Performance) ทั้ง Binary Search และ AVL Tree ใช้เวลาในการค้นหาใกล้เคียงกันมากที่ระดับประมาณ 18 นาโนวินาที (0.000018 ms) ณ ข้อมูล 100,000 รายการ ซึ่งเร็วกว่า Sequential Search ถึง 1,290 เท่า

เหตุผลที่ระบบลงทะเบียนเรียนต้องใช้ AVL Tree แทน Sorted Array:

1. ในระบบการลงทะเบียนเรียน รายวิชามีการเปลี่ยนแปลงตลอดเวลา ได้แก่ การเปิดรายวิชาใหม่ (Insert/Add) และการปิดหรือถอนรายวิชา (Delete/Drop)
2. หากใช้ Sorted Array + Binary Search: การค้นหาจะทำได้ $O(\log n)$ ก็จริง แต่การเพิ่มหรือลบวิชาจะต้องเลื่อนย้ายสมาชิกใน Array ทั้งก้อน ซึ่งกินเวลาถึง $O(n)$
3. การเลือกใช้ AVL Tree ทำให้ระบบสามารถรองรับทั้ง การค้นหา (Search), การเพิ่มวิชา (Insert) และการถอนวิชา (Delete) ได้ในเวลา $O(\log n)$ เสมอทุกกรณี ระบบจึงไม่เกิดอาการกระตุกหรือค้างแม้ข้อมูลรายวิชาจะขยายตัวขึ้นในอนาคต

บทที่ 6

6. สรุปผลการพัฒนาและปัญหาอุปสรรค

6.1 สรุปผลสัมฤทธิ์ของโครงการ (Project Achievements)

โครงการระบบจัดการและจำลองการลงทะเบียนเรียนเรียนอัจฉริยะ (KU CSC Course Manager & Scheduler) สามารถบรรลุวัตถุประสงค์ที่ตั้งไว้ได้อย่างครบถ้วน โดยประสบความสำเร็จในการบูรณาการองค์ความรู้ด้านโครงสร้างข้อมูลนามธรรม (Abstract Data Types) และขั้นตอนวิธี (Algorithms) มาประยุกต์ใช้แก้ปัญหาจริงได้อย่างมีประสิทธิภาพ การทำงานร่วมกันของโครงสร้างข้อมูลและอัลกอริทึมขั้นสูง ได้แก่ AVL Tree, Trie, 2D Time Grid, Kahn's Topological Sort และ Branch & Bound Backtracking ส่งผลให้ระบบมีความแม่นยำ ประมวลผลได้อย่างรวดเร็ว ปราศจากข้อผิดพลาด และพร้อมสำหรับการพัฒนาต่อยอดเพื่อนำไปใช้งานจริง

6.2 ปัญหาอุปสรรคที่พบและแนวทางแก้ไข (Challenges & Solutions)

1. ปัญหาหน่วยความจำรั่วไหล (Memory Leak) และ Segmentation Fault ในภาษา C

ปัญหา: การจองหน่วยความจำแบบพลวัต (Dynamic Memory Allocation) จำนวนมากสำหรับสร้างโหนดในโครงสร้าง AVL Tree, Trie และกราฟ ทำให้เสี่ยงต่อการเกิดหน่วยความจำรั่วไหลและข้อผิดพลาดในการเข้าถึงหน่วยความจำ หากไม่มีการจัดการคืนค่าที่ถูกต้อง

แนวทางแก้ไข: ออกแบบฟังก์ชันสำหรับทำความสะอาดหน่วยความจำเฉพาะทาง (เช่น `avlFree`, `trieFree`, `graphFree` และ `freeCourseNodes`) โดยใช้เทคนิคการคืนค่าหน่วยความจำแบบเรียกตัวเอง (Recursive Deallocation) เพื่อล้างข้อมูลทุกครั้งก่อนปิดโปรแกรมหรือรีเซ็ตระบบ

2. ปัญหาการประมวลผลระบบแนะนำรายวิชาที่ใช้เวลานาน (Combinatorial Explosion)

ปัญหา: การค้นหาชุดรายวิชาที่เหมาะสมที่สุดด้วยวิธี Brute-force มีความซับซ้อนด้านเวลาพุ่งสูงในระดับเอ็กซ์โพเนนเชียล (Exponential Time Complexity เช่น $O(2^k)$) ส่งผลให้โปรแกรมหยุดการตอบสนองเมื่อมีจำนวนรายวิชาเปิดสอนในระบบเป็นจำนวนมาก

แนวทางแก้ไข: ประยุกต์ใช้เทคนิค Branch and Bound Pruning โดยระบบจะประเมิน "หน่วยกิตสูงสุดที่เป็นไปได้ในอนาคต" หากพบว่าผลรวมเส้นทางนั้นไม่สามารถเอาชนะค่าที่ดีที่สุด (Best) ที่เคยบันทึกไว้ได้ ระบบจะทำการตัดกิ่ง (Pruning) ที่ทันที ซึ่งช่วยลดจำนวนรอบการประมวลผลลงได้มากกว่า 99%

3. ปัญหาการแสดงผลภาษาไทยและอักขระพิเศษบน Windows Console

ปัญหา: ไลบรารี ncurses รุ่นมาตรฐานบนระบบปฏิบัติการ Windows ไม่รองรับการตัดคำ และการแสดงผลอักขระแบบ UTF-8 อย่างสมบูรณ์ ทำให้โครงสร้างส่วนต่อประสาน (UI) และเส้นตารางเกิดการบิดเบี้ยว

แนวทางแก้ไข: เปลี่ยนมาคอมไพล์ด้วยชุดไลบรารี ncursesw (Wide-character version) บนมาตรฐานสภาพแวดล้อม MSYS2 UCRT64 พร้อมทั้งพัฒนาฟังก์ชัน `get_safe_slice_len` เพื่อคำนวณความกว้างของอักขระ UTF-8 แบบหลายไบต์อย่างปลอดภัย ทำให้การแสดงผลบนหน้าจอเทอร์มินัลถูกต้องและสวยงาม

6.3 ข้อเสนอแนะสำหรับการพัฒนาต่อยอด (Future Work)

ระบบเชื่อมโยงข้อมูลปฏิทิน (Calendar Synchronization): พัฒนาระบบเชื่อมโยงตารางเรียนร่วมกับ Google Calendar API ผ่านระบบ Webhook เพื่ออำนวยความสะดวกให้นักศึกษาสามารถส่งออกตารางเรียนที่จัดเสร็จแล้วไปยังสมาร์ทโฟนได้โดยตรง

ยกระดับระบบแนะนำตารางเรียนส่วนบุคคล (Personalized Scheduling): พัฒนาระบบแนะนำรายวิชาด้วยอัลกอริทึม A* Search หรือ Constraint Satisfaction Problem (CSP) ร่วมกับเทคโนโลยี Machine Learning เพื่อรองรับการจัดตารางเรียนตามความพึงพอใจและรูปแบบการใช้ชีวิตของนิสิต (เช่น การตั้งเงื่อนไขหลีกเลี่ยงการเรียนคาบเช้า หรือการจัดตารางให้มีวันหยุดในวันศุกร์)